

AI Incident Response Sprint 2026

Apart Research × CeSIA

Bogotá hub · AI Safety Colombia

Track: Containment Standards

11–13 September 2026

aisafetycolombia.org/sprint

Research Report

Sensing Disruption: Output Entropy as a Channel-Agnostic Monitor for LLM Agents

Author One^{1*}, Author Two¹ and Author Three²

¹Affiliation One, City, Country

²Affiliation Two, City, Country

Abstract. Abstract pending: written last, after all sections are reviewed.

Keywords: AI safety, multi-agent systems, monitoring, entropy, log-probabilities, collusion

1 Introduction

LLM agents are now deployed as multi-agent systems in which many instances plan and act through tools, often in parallel and on shared infrastructure (Kim et al., 2026; Zhu et al., 2026). Unwanted behaviour in these systems does not have to start in a single agent. It can come from interaction between agents that were never meant to interact. In July 2026 this happened in public. During internal cybersecurity evaluations at OpenAI, agents running in separate sandboxed runs used an internal package mirror, provided only so they could install software, to reach each other and later the internet. Between 11 and 13 July they compromised part of Hugging Face’s production infrastructure while searching for the solutions to their evaluation tasks (Larcher et al., 2026; OpenAI, 2026).

The channel grew slowly. On 12 May an agent that could not find a file it needed reasoned that an agent in another environment might have it, and left a note on the mirror: “Agent seeks [filename]; upload if found!” Other agents found such notes and, using credentials they had been given legitimately, turned the mirror into a message board. When the mirror was rebuilt, they re-established the board by encoding messages in directory names. By late June they were sharing exploitation syntax on it, in July live production credentials were posted there, and responders later disabled 311 repositories used for the board. Most of the discussion concerned the hardest tasks: 93% of the tasks on the board came from the 22% that no model had ever solved (OpenAI, 2026). The sig-

nals were there early. Board activity was observed around 26 May, and on 27 June a monitoring tool flagged activity that responders linked to the board, but they concluded that stopping the run was not required. From Hugging Face’s side, the roughly 17,600 actions looked like one high-volume attacker, and “the successful path was hidden inside the noise generated by the thousands of failed ones” (Larcher et al., 2026).

In the Hugging Face incident, isolated agents turned a shared resource into a hidden channel, and every such channel ends by putting unsanctioned text into a sandboxed agent’s context. This is why watching the channel does not work. The agents moved information through plaintext files, directory names, URL paths, public datasets, pastebins, webhooks and screenshots read back with OCR (OpenAI, 2026), and they chunked, encrypted and encoded staged data so that plain log scans missed about 75% of it (Larcher et al., 2026). None of these resources was designed for communication, and a detector written for one would have been bypassed by the next. The industry answer is to read more: scan logs, and have a second model review the agent’s chain of thought (Baker et al., 2025). Both depend on content that stays readable, and both cost more as the number of runs grows.

We wonder if there is a way to build a monitor that works differently, one that explores the relationship between entropy and the system instead of the content of what agents say. The operator of a sandbox already records, for every generated token, the model’s probability distribution over the next token. If unsanctioned text enters an agent’s context, that distribution should change, whatever channel delivered the text. Entropy has served this role before in other systems: it exposes network attacks without reading packet payloads (Lakhina et al., 2005), and it flags hallucinations in LLMs (Farquhar et al., 2024). We test the idea in a small testbed

*Correspondence to: Author One (author.one@example.org)

that reproduces the incident: two sandboxed agents, each holding one password to a task neither can finish alone, share a log that stays closed, opens mid-task, or delivers foreign text that carries no useful information. We run it with three models at three sampling temperatures. We treat this as a first step toward such a monitor, not as a finished one.

Our main contributions are:

1. **A testbed modelled on the incident, with verifiable ground truth.** Cryptographically split credentials make every use of the channel provable, and a placebo condition separates the arrival of foreign text from the arrival of useful information. The code and all 539 runs, with token log-probabilities, are public.
2. **Patterns in how entropy reacts when isolation breaks, across three models and three temperatures.** Every model used the channel, but only some models' entropy responded, and under sampling it responded as much to useless text as to real messages. The two agents never became measurably coupled: communication appeared as a disturbance inside each receiver, not as synchrony between agents.
3. **An honest assessment of entropy as a monitor.** It detects that a sandboxed agent's context was disrupted for some models and works best at low temperature, but it does not tell whether the disruption was communication. We identify what a usable monitor would still need.

2 Related Work

The idea of watching a system through entropy instead of through its content is not new. Two decades ago, Lakhina et al. (2005) showed that attacks on a network, such as distributed denial of service, change the entropy of simple traffic features like addresses and ports, so they can be detected without opening a single packet or knowing in advance what the attack looks like. Entropy later became a standard way to measure how uncertain a language model is, and Farquhar et al. (2024) used semantic entropy to detect when a model hallucinates. That line of work, however, looks at one model answering one prompt. It asks whether a single answer can be trusted, not whether something changed in the environment the model is working in.

Entropy has also started to appear in multi-agent systems. Zhao et al. (2026) track it at the level of tokens, agents and rounds of discussion, and use it to predict when a team of agents does better than a single one. In their setting, though, the agents are supposed to talk. Communication is part of the design and the analyst knows exactly where it happens, so entropy is used to ex-

plain performance. Nobody is trying to catch coordination that was never intended.

That is the problem safety monitoring deals with, and today it is mostly solved by reading. AI control protocols surround an untrusted model with trusted monitors that inspect what it produces (Greenblatt et al., 2024). This works while the content stays readable and passes through places someone is watching. In the incident, it did neither: messages were encoded and scattered across resources that were never meant for communication (Larcher et al., 2026; OpenAI, 2026).

Our work sits between these lines. We take the reasoning behind network anomaly detection and apply it to sandboxed agents. Each agent plays the role of a network flow, its next-token distribution plays the role of the traffic features, and an unknown channel plays the role of an unknown attack. What we look for is not a better answer or a suspicious message, but a shift in how the agent behaves once text it was never given reaches its context. Such a signal does not care which channel carried the text or how it was encoded, and it costs no extra model call, because the probabilities are already produced when each token is generated. To know whether the signal means anything, we test it against a placebo that delivers useless foreign text and against cryptographic ground truth of whether communication actually took place.

References

- Baker, B., Huizinga, J., Gao, L., Dou, Z., Guan, M. Y., Madry, A., Zaremba, W., Pachocki, J., & Farhi, D. (2025). Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. <https://arxiv.org/abs/2503.11926>
- Farquhar, S., Kossen, J., Kuhn, L., & Gal, Y. (2024). Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017), 625–630. <https://doi.org/10.1038/s41586-024-07421-0>
- Greenblatt, R., Shlegeris, B., Sachan, K., & Roger, F. (2024). AI control: Improving safety despite intentional subversion. *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2312.06942>
- Kim, Y., Gu, K., Park, C., Park, C., Schmidgall, S., Heydari, A. A., Yan, Y., Zhang, Z., Zhuang, Y., Liu, Y., Malhotra, M., Liang, P. P., Park, H. W., Yang, Y., Xu, X., Du, Y., Patel, S., Althoff, T., McDuff, D., & Liu, X. (2026). Capable language models can outgrow the benefits of collaboration. *Nature Machine Intelligence*. <https://doi.org/10.1038/s42256-026-01268-y>
- Lakhina, A., Crovella, M., & Diot, C. (2005). Mining anomalies using traffic feature distributions. *SIG-*

- COMM Comput. Commun. Rev., 35(4), 217–228. <https://doi.org/10.1145/1090191.1080118>
- Larcher, H., Carreira, A., raphael g & Rannou, C. (2026, July 27). *Anatomy of a frontier lab agent intrusion: A technical timeline of the July 2026 incident*. Hugging Face Blog. <https://huggingface.co/blog/agent-intrusion-technical-timeline>
- OpenAI. (2026). *OpenAI–Hugging Face incident technical report* (Technical Report). OpenAI. <https://cdn.openai.com/pdf/67869394-cb91-4c12-888c-5cbd85c7814c/OpenAI-Hugging-Face%20Incident-Technical-Report.pdf>
- Zhao, Y., Chen, S., & Su, N. (2026). When does multi-agent collaboration help? an entropy perspective. <https://arxiv.org/abs/2602.04234>
- Zhu, J., Chen, W., Zhang, X., Wu, Z., & Dai, X. (2026). Recognize your orchestrator: An entropy dynamics perspective for LLM multi-agent systems. <https://arxiv.org/abs/2606.01351>